

**SANGOLA MAHAVIDYALAYA SANGOLA DEPARTMENT OF
COMPUTER SCIENCE AND APPLICATION
B.C.A, B.Sc(CS),B.Sc(ECS)- III
Subject – Dot Net Core**

Practical /Journal Assignment-02

1. Create a class `Book` that automatically sets a default title and author when no information is provided. Demonstrate this using a default constructor.
2. Design a class `Student` that stores roll number and name. Implement a copy constructor to create a duplicate student record from an existing object.
3. Build a class `Car` where details like brand, model, and price can be set at the time of object creation using a parameterized constructor.
4. Create a class `Employee` that allows creation of an employee object either with only a name, or with name and salary, using constructor overloading.
5. Implement a class `Bank` that has a static constructor to initialize a bank's name and branch code before creating any customer accounts.
6. Write a method `BookTicket` that accepts passenger name, destination, and seat type using named parameters for clarity.
7. Create a method `ApplyDiscount` that takes product price by reference and reduces it by a given percentage.
8. Implement a method `DivideNumbers` that accepts two integers and returns both quotient and remainder using out parameters.
9. Write a method `SUM` which accept Default Parameters”.
10. Create a method `ADD` that can accept dynamic data like number,string,float,character.
11. Implement a method `SquareNumber` that takes an integer value as parameter and returns its square.
12. Write a method `AddMarks` that can accept any number of subject marks and return the total using params array.
13. Create a `struct Product` with fields for id, name, and price. Display details of products entered by the user.
14. Create a base class `Person` with name and age, and a derived class `Student` with roll number and marks. Demonstrate single inheritance.
15. Implement classes `Student->Exam->Result` to calculate percentage, Total Marks and Grade secured by student.
16. Build a base class `Account` and derive two classes `SavingsAccount` and `CurrentAccount`. Demonstrate hierarchical inheritance.

17. Design interfaces 'ISports' and 'ICultural'. A class 'Student' should implement both to show achievements in sports and cultural activities.
18. Create an interface with a default interface method.
19. Implement a class 'Laptop' with properties for brand and price. Allow reading and modifying these properties.
20. Implement a class 'Order' that generates an 'OrderId' which can only be read but not modified.
21. Implement a class 'PasswordManager' with a property 'Password' that can only be set but not retrieved.
22. Create a class 'Library' that stores an array of book titles. Access books using an indexer.
23. Create a class 'Calculator' with methods 'Add(int a, int b)', 'Add(float a, float b)' and 'Add(double a, double b, double c)' demonstrating method overloading.
24. Overload the unary minus (-) operator for a class 'Temperature' to convert a positive Celsius value into negative (representing below zero).
25. Overload the minus operator for a class 'Point' to subtract coordinates of two points.
26. Overload the binary plus operator for a class 'ComplexNumber' to add two complex numbers.
27. Overload the '>' and '<' operators for a class 'Box' based on volume comparison.
28. Overload the increment (++) and decrement (--) operators for a class 'Counter'.
29. Create a base class 'Shape' with a method 'Draw()'. Override it in derived classes 'Circle' and 'Rectangle'.
30. Define an abstract class 'Appliance' with abstract method 'TurnOn()'. Implement it in classes 'Fan' and 'Light'.
31. Create a sealed class 'SecuritySystem' that cannot be inherited, containing a method 'Authenticate()'.